

UNIVERSIDADE DA CORUÑA

Programación de Sistemas

2.4 Layouts

Carlos Vázquez Regueiro
carlos.vazquez.regueiro@udc.es

Índice

1. **Views**
2. Librerías de terceros
3. Ejemplos de layouts
4. CustomViews
5. RecyclerView

Views (I)

- Las Views representan vistas en la pantalla del dispositivo
- Las activities/fragments son quienes indican qué layout se renderiza
  - Se pueden instanciar mediante código Java
  - Normalmente se realizan mediante XML, los layouts

```
<TextView
    android:id="@+id/hello_world"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello World!" />
```

- *layout_width* y *layout_height* obligatorios para todas las views
- *id* es el identificador de la vista para recogerla en la Activity



Views (II)

- En la Activity las vistas se recogen de la siguiente manera

```
TextView textView = (TextView) findViewById(R.id.hello_world);
```

- Desde Android 8 no hace falta el downcasting

```
TextView textView = findViewById(R.id.hello_world);
```

- Con kotlin

```
import kotlinx.android.synthetic.main.activity_main.*
       [...]
override fun onCreate(savedInstanceState: Bundle?) {
   super.onCreate(savedInstanceState)
   setContentView(R.layout.activity_main)
   hello_world.text = "Hello Kotlin!"
```

Views (III)

- Los *ViewGroups* son *Views* invisibles que agrupan *Views*
  - Sirven para organizar las vistas en la pantalla

```
<LinearLayout
   android:id="@+id/about_layout"
   android:layout_width="match_parent"
   android:layout_height="wrap_content"
   android:background="@color/primary_color"
   android:orientation="vertical"
   android:padding="@dimen/card_double_padding">

   <ImageView
       android:layout_width="wrap_content"
       android:layout_height="wrap_content"
       android:src="@drawable/ic_launcher_big"/>

   <TextView
       android:id="@+id/tv_about_title"
       android:layout_width="wrap_content"
       android:layout_height="wrap_content"/>
</LinearLayout>
```



Views (IV)

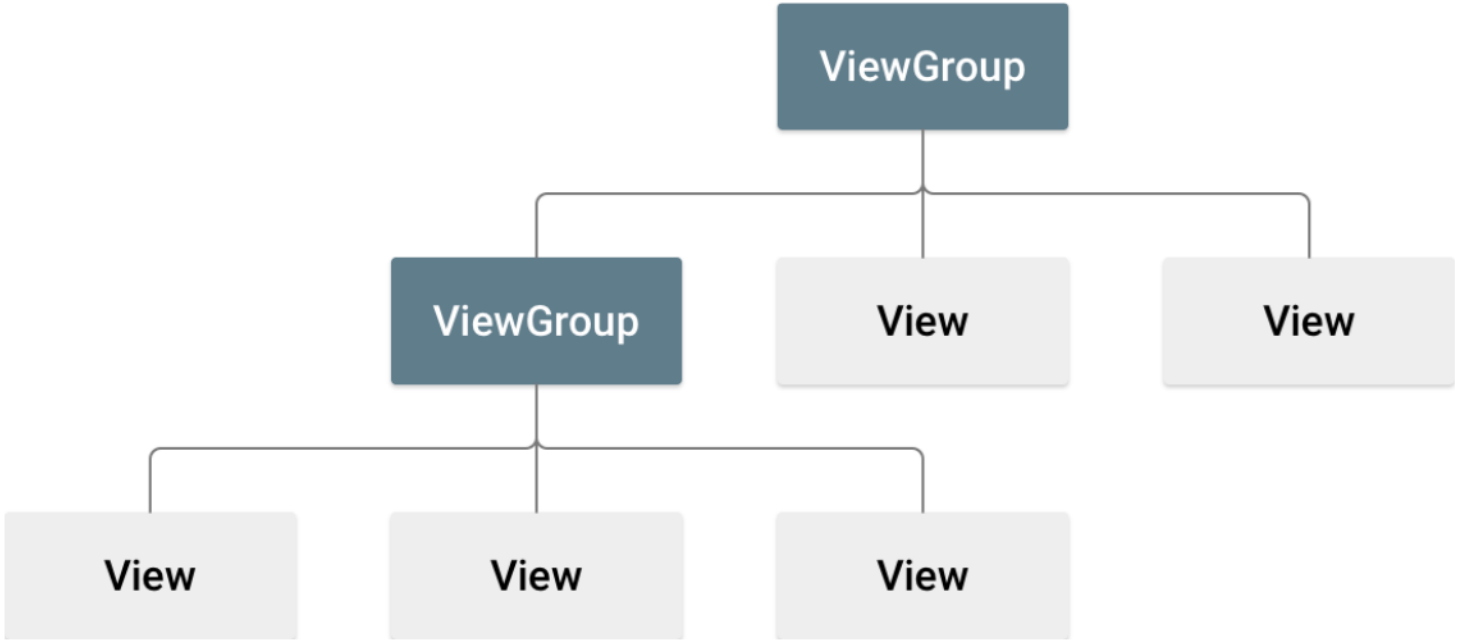
Views

- TextView
- ImageView
- EditText
- Button
- CheckBox
- RadioButton
- Spinner
- ProgressBar

ViewGroups

- FrameLayout
- LinearLayout
- RelativeLayout
- CoordinatorLayout
- ConstraintLayout
- RecyclerView
- ViewPager
- ScrollView

Views (IV)



Views (V)

- Cada vista debe proporcionar:
  - android:id - Identificador de la vista (opcional)
  - android:layout_height - Alto de la vista (obligatorio)
  - android:layout_width - Ancho de la vista (obligatorio)
- Values of fill model:
  - match_parent - ocupa todo lo que ocupa el padre (fill_parent, hasta API 8)
  - wrap_content - ocupa lo que ocupa el contenido
  - No se suelen poner tamaños fijos (100px, por ejemplo) porque no se adaptan

Views (y VI)

- Cada clase ViewGroup implementa una clase anidada que extiende ViewGroup.LayoutParams
- Atributos de diseño más comunes en las vistas:
  - margin, padding, background, color
  - En otros temas hablaremos de los *styles* y su correcta organización
- Visibility
  - VISIBLE - se muestra la View
  - INVISIBLE - se oculta la View pero sigue estando en el layout (ocupa espacio)
  - GONE - se borra la View del layout (no ocupará su espacio; ojo diseño relativos)

Índice

1. Views
2. **Librerías de terceros**
3. Ejemplos de layouts
4. CustomViews
5. RecyclerView

Librerías de terceros

- Antes de hacerlo, búscalo
- Implementaciones complejas (acceso a red)
- Componentes y comportamientos “no estándares” (custom views)
- Animaciones
- Evitar *boiler plate*
- Tanto empresas (Gooogle, Facebook, AirB&B, Square, etc) como desarrolladores anónimos publican librerías que pueden ser de ayuda
- Fuente principal de facto: Github
- Ojo con las licencias
  - Aunque muchas son open source, requieren atribución

Vinculación de vistas (I)

- La vinculación de vistas (View bindings) sustituye a findViewById
- Genera una clase de vinculación que se usa para referenciar elementos de la vista
  - Válido tanto en actividades como fragmentos
  - El nombre de la clase de vinculación está asociado al nombre del layout (!!)
- Se debe habilitar en el fichero de configuración del gradle del módulo

```
buildFeatures {
   viewBinding = true
}
```

Vinculación de vistas (II)

- Vinculación en actividades

```
private ActivityMainBinding binding;
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    binding = ActivityMainBinding.inflate(getLayoutInflater());
    View view = binding.getRoot();
    setContentView(view);

    binding.tvHello.setText(" Hello! ");
    binding.butNew.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View view) {
            // TODO
        }
    });
}
```

Vinculación de vistas (y III)

- Vinculación en fragmentos

```
private FragmentDownBinding binding;

@Override
public View onCreateView (LayoutInflater inflater,
                          ViewGroup container,
                          Bundle savedInstanceState) {
    binding = FragmentDownBinding.inflate(inflater, container, false);
    View view = binding.getRoot();
    return view;
}
@Override
public void onDestroyView() {
    super.onDestroyView();
    binding = null;
}
```

Índice

1. Views
2. Librerías de terceros
3. **Ejemplos de layouts**
4. CustomViews
5. RecyclerView

Linear Layout

- Diseño Lineal
- ViewGroup que alinea todas sus vistas en una única dirección
  - android:orientation = [vertical|horizontal] (obligatorio)
- Compatible con la asignación de un volumen a campos
- Distribución equitativa
  - android:layout_height = "0dp" (para un diseño vertical)
  - android:layout_width = "0dp" (para horizontal)
  - android:layout_weight = "1"
  - android:layout_weightSum -> en el ViewGroup que agrupa las vistas



Relative Layout

- Diseño Relativo
- ViewGroup que alinea todas sus vistas con una posición relativa a otras vistas
  - android:layout_alignParentTop
  - android:layout_centerVertical
  - android:layout_toRightOf
  - android:layout_below
  - …



Constraint Layout

- Diseño relativo
  - Más flexible que el RelativeLayout
- Mejor adaptación al diseño de interfaces mediante herramientas de diseño
  - Las otras vistas es más fácil hacerlas por XML
  - Esta es más fácil mediante interfaz gráfica
- Similar al Auto Layout de Swift (iOS)

Coordinator Layout

- Se utiliza como layout top-level layout
  - El primero de la jerarquía
- Gestiona comportamientos, interacciones, animaciones, entre el resto de vistas
- Se utiliza cuando tenemos vistas complejas:
  - AppBarLayout
  - TabLayout
  - Navigation Drawer
  - Bottom Navigation

Índice

1. Views
2. Librerías de terceros
3. Ejemplos de layouts
4. **CustomViews**
5. RecyclerView

Custom Views (I)

- Los desarrolladores pueden crear sus propios ViewGroups
  - Ayudan a la organización
  - Reutilización de código ante vistas que se emplean a lo largo de la aplicación
- La implementación extenderá del View o ViewGroup más adecuado
  - Se buscará aquel que más se parezca al nuevo que se quiere crear
  - Objetivo: aprovechar sus características
- Normalmente, se renderizará un layout
  - Aunque se pueden instanciar y colocar las vistas desde la clase

Custom Views (II)

```
<LinearLayout android:id="@+id/call_us_content"
   xmlns:android="http://schemas.android.com/apk/res/android"
   android:layout_width="match_parent"
   android:layout_height="wrap_content"
   android:background="@color/white"
   android:gravity="center_horizontal"
   android:orientation="vertical"
   android:padding="16dp">

   <TextView
       android:id="@+id/tv_call_us_title"
       style="@style/CallUs.Title"
       android:layout_width="wrap_content"
       android:layout_height="wrap_content"
       android:gravity="center" />

   <Button
       android:id="@+id/bt_call_us_now"
       style="@style/Button.CallUs"
       android:layout_width="wrap_content"
       android:layout_height="wrap_content"
       android:text="@string/unsuccess_action_call_us_now" />
</LinearLayout>
```



Custom Views (III)

```
public class CallUsView extends LinearLayout {

    @BindView(R.id.call_us_content)
    View mContent;
    @BindView(R.id.tv_call_us_title)
    TextView mTvTitle;
    private String mTitle;
    private String mPhoneNumber;

    public CallUsView(Context context) {
        super(context);
        init();
    }

    public CallUsView(Context context,
                      @Nullable AttributeSet attrs) {
        super(context, attrs);
        TypedArray a = context.getTheme()
                .obtainStyledAttributes(attrs,
                        R.styleable.CallUsView, 0, 0);
        try {
            mTitle = a.getString(
                    R.styleable.CallUsView_title);
        } finally {
            a.recycle();
        }
        init();
    }
```

```
    private void init() {
        LayoutInflater inflater = LayoutInflater.from(
                                        getContext());
        View view = inflater.inflate(
                    R.layout.view_call_us, this, true);
        ButterKnife.bind(this, view);
        mTvTitle.setText(mTitle);
    }

    public void setPhoneNumber(String phoneNumber) {
        mPhoneNumber = phoneNumber;
    }

    @OnClick(R.id.bt_call_us_now)
    void onCallUsNowClick() {
        if (!TextUtils.isEmpty(mPhoneNumber)) {
            Intent intent = new
Intent(Intent.ACTION_DIAL);
            intent.setData(Uri.parse("tel:" +
                    mPhoneNumber.replace(" ", "")));
            getContext().startActivity(intent);
        }
    }
}
```

Custom Views (y IV)

- Una vista puede llegar a ser muy compleja
- Buscar en repositorios públicos
  - Github
  - AndroidArsenal
- Ejemplo:
  - Cliente quería el selector de fecha de iOS
  - Estimación alta: inabordable hacerlo dentro del proyecto
  - Búsquedas de alternativas
  - SingleDateAndDatePicker

Índice

1. Views
2. Librerías de terceros
3. Ejemplos de layouts
4. CustomViews
5. **RecyclerView**

Recycler View (I)

- Utilizado cuando tenemos una lista de elementos
  - Cuando son muchos
  - Cuando estos cambian con frecuencia
- Antes existían vistas como ListView o GridView
  - Poco eficientes
- RecyclerView “recicla” vistas
  - Cambia el contenido pero del mismo objeto en memoria
  - Hay que tener cuidado con esta circunstancia

Recycler View (II)

- En el model de RecyclerView participan:
  - RecyclerView: contenedor principal
  - RecyclerView.ViewHolder: vistas de cada item
  - LinearLayoutManager: gestor de diseño
    - Puedes ser también un GridLayoutManager
    - O puede ser uno propio (custom)
  - RecyclerView.Adapter: adaptador del contenido

Recycler View (III)

- Cada ViewHolder es responsable de mostrar un elemento individual con una vista
  - Por ejemplo, lista de mis discos, cada ViewHolder representa un álbum individual
- El RecyclerView crea solamente la cantidad de ViewHolder que sean necesarios para que se vean en pantalla, más algunos adicionales
- A medida que el usuario se desplaza por la lista, el RecyclerView toma las vistas fuera de pantalla y vuelve a vincularlas con los datos que se desplazan en la pantalla
- Va reciclando vistas cambiando el contenido

Recycler View (y IV)

- El Adapter crea contenedores de vistas según sea necesario
- Los vincula con sus datos
  - Asigna el contenedor de vistas a una **posición**
  - Llama al método onBindViewHolder() del adaptador.
    - Este método usa la posición del contenedor de vistas para determinar cuál debería ser el contenido, en función de su posición en la lista.
- Ante cambios en los elementos que se muestran hay que llamar al

  RecyclerView.Adapter.notify…() adecuado
  - notifyDataSetChanged, notifyItemChanged, notifyItemInserted, ...

Ejemplo - Activity Layout

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_height="match_parent"
    android:layout_width="match_parent"
    android:orientation="vertical">

    <androidx.swiperefreshlayout.widget.SwipeRefreshLayout
        android:id="@+id/swipe_refresh"
        android:layout_width="match_parent"
        android:layout_height="match_parent">

        <androidx.recyclerview.widget.RecyclerView
            android:id="@+id/categories_rv"
            android:layout_height="wrap_content"
            android:layout_width="match_parent" />

    </androidx.swiperefreshlayout.widget.SwipeRefreshLayout>

</LinearLayout>
```

Ejemplo - Activity (I)

```
public class MainActivity extends AppCompatActivity {

    RecyclerView recyclerView;
    private ArticlesAdapter mAdapter;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        recyclerView = findViewById(R.id.categories_rv);

        ArrayList<Article> initialData = new ArrayList<>();
        for (int i=0; i< 10; i++)
            initialData.add(new Article(i));
        initRecycler(initialData);
    }
```

Ejemplo - Activity (y II)

```
private void initRecycler(ArrayList<Article> articles) {
        mAdapter = new ArticlesAdapter (articles);
        LinearLayoutManager linearLayoutManager =
                new LinearLayoutManager(this, LinearLayoutManager.VERTICAL, false);
        recyclerView.setLayoutManager(linearLayoutManager);
        recyclerView.setAdapter(mAdapter);
    }
}
```

Ejemplo - ArticlesAdapter (I)

```
public class ArticlesAdapter extends RecyclerView.Adapter<ArticlesAdapter.MyViewHolder> {

    private final ArrayList<Article> mDataset;

    public static class MyViewHolder extends RecyclerView.ViewHolder {
        public TextView title;
        public MyViewHolder(View view) {
            super(view);
            title = view.findViewById(R.id.tile_title_tv);
        }
        public void bind(Article article) {
            title.setText(article.getTitle());
        }
    }

    public ArticlesAdapter(ArrayList<Article> myDataset) {
        mDataset = myDataset;
    }
```

Ejemplo - ArticlesAdapter (y II)

```
    @NonNull
    @Override
    public ArticlesAdapter.MyViewHolder onCreateViewHolder(ViewGroup parent,
                                                           int viewType) {
        View v = LayoutInflater.from(parent.getContext())
                .inflate(R.layout.article_tile, parent, false);
        return new MyViewHolder(v);
    }

    @Override
    public void onBindViewHolder(MyViewHolder holder, int position) {
        holder.bind(mDataset.get(position));
    }

    @Override
    public int getItemCount() {
        return mDataset.size();
    }
}
```

Eventos en un Recycler View (I)

- El RecyclerView no tiene asociado un evento cuando se pulsa un elemento (como el ListView), por lo que tenemos que generarlo y gestionarlo nosotros

1. Definir una interfaz (típicamente en el Adapter)

```
public interface OnItemClickListener {
   public void onClick(View view, int position);
}
```

2. En la clase del Adapter definir una variable y una función para guardar el listener:

```
private static OnItemClickListener clickListener;
public void setClickListener(OnItemClickListener itemClickListener) {
   this.clickListener = itemClickListener;
}
```

Eventos en un Recycler View (II)

3. En la clase ViewHolder del adaptador indicar que implementa View.OnClickListener, asociar listener a cada elemento creado (view) y gestionar el evento (onClick):

```
public static class MyViewHolder extends RecyclerView.ViewHolder
                                          implements View.OnClickListener {
    public TextView title;
    public MyViewHolder(View view) {
        super(view);
        title = view.findViewById(R.id.tile_title_tv);
        view.setOnClickListener(this);
    }
       [...]
    @Override
    public void onClick(View view) {
        if (clickListener != null) clickListener.onClick(view, getAdapterPosition());
    }
}
```

Eventos en un Recycler View (y III)

4. En la actividad ya se puede definir el nuevo listener declarado en el adaptador y gestionar el evento (método onClick):

```
mAdapter.setClickListener(new ArticlesAdapter.OnItemClickListener() {
    @Override
    public void onClick(View view, int position) {
        Log.d("_TAG", " Item " + position );
        Toast.makeText(getApplicationContext(), "item " + position,
                Toast.LENGTH_SHORT).show();
    }
});
```

Referencias

- https://developer.android.com/guide/topics/ui/declaring-layout
- https://developer.android.com/guide/topics/ui/layout/linear
- https://developer.android.com/guide/topics/ui/layout/relative
- https://developer.android.com/training/constraint-layout
- https://developer.android.com/guide/topics/ui/layout/recyclerview
- https://developer.android.com/topic/libraries/view-binding